```
$ git log --grep="oops" | wc -l
```

I Kept Breaking Things

Four accidental patterns for durable AI, courtesy of my mediocre code

Kevin Burns · SR Developer Advocate · Uber

```
cadence-ai-samples  ·  built on Cadence
```

Follow along

github.com/cadence-workflow/cadence-ai-samples


I Kept Breaking Things
Four accidental patterns for durable AI, courtesy of my mediocre code.
Thank you for spending your evening here
instead of doing literally almost anything else.

SHOW OF HANDS

Who's built an agentic AI workflow?

What broke?

THE PROBLEM

AI demos are easy. AI that keeps working is hard.

1 **Flaky calls**

APIs time out, rate-limit and fail mid-run.

2 **Lost progress**

A crash means re-running everything.

3 **Waiting on humans and time**

Approvals and schedules live in cron glue.

4 **Black-box runs**

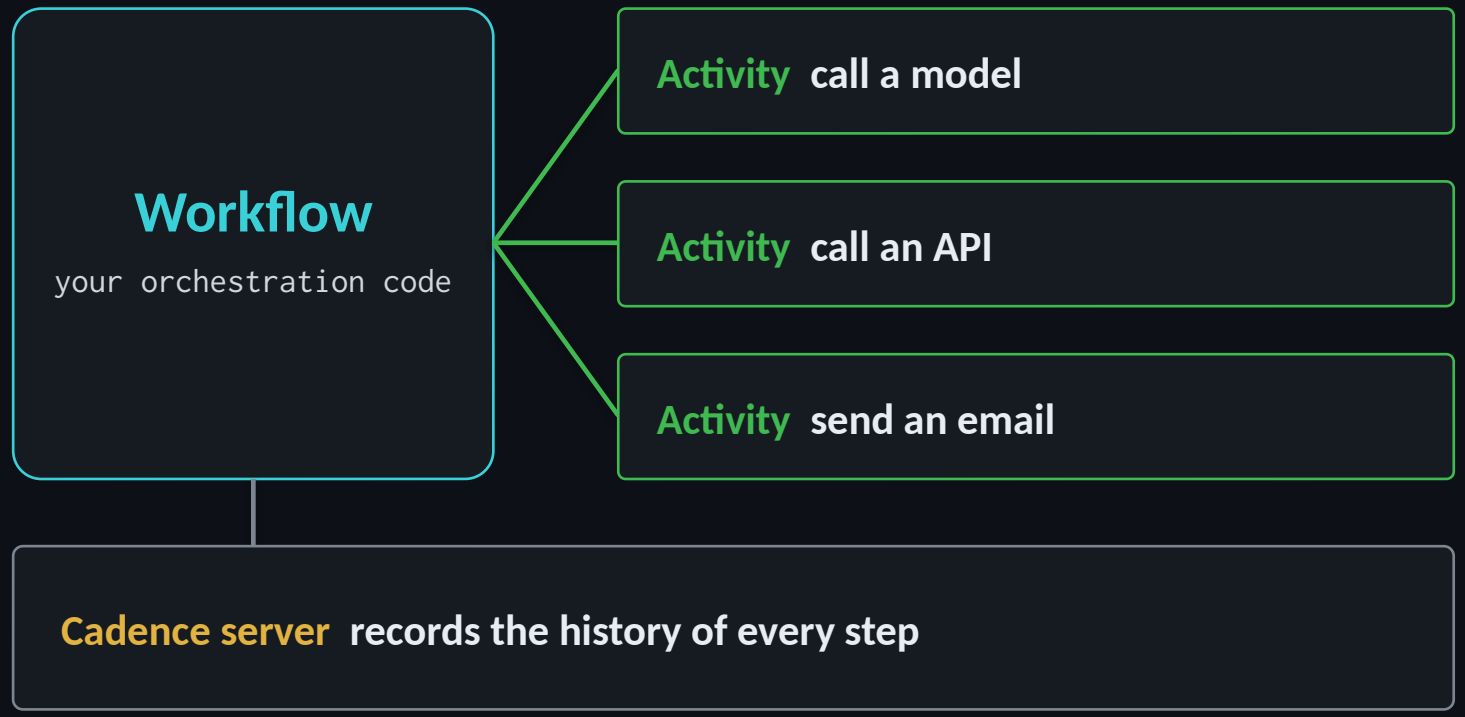
What did the agent do? Why did it fail?

Cadence already solves all four. Here's how.

CONTEXT

What is Cadence?

Open-source durable workflow orchestration, born at Uber.



worker crashes → replay history → Workflow resumes

Workflow
Durable code that survives crashes.

Activity
Outside calls, retried and timed out.

Signal
Send data into a running Workflow.

Cadence Web
Every step, visible in the browser.

CONTEXT

Why Cadence for Agentic AI workflows?

AI calls are slow, flaky and expensive. Your workflows don't have to be.

1 · Retries & timeouts

Every LLM and tool call is a retried Activity.

2 · Durable state

Crash or deploy? It resumes. No re-runs.

3 · Signals & timers

Wait days for approvals with zero workers.

4 · History & Cadence Web

Every step, input and output, visible.

AGENT LOOP

Every step runs as a Cadence Activity.



Built in: cadence-python-client contrib for Google ADK + OpenAI Agents SDK

CONTEXT

Meet Jev: decisions, not paragraphs

a chatty LLM

Ticket: “I was charged twice this month!”

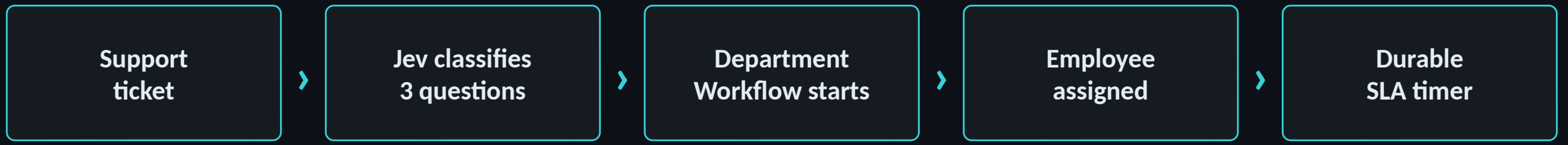
“This sounds like it could be a billing problem, probably fairly urgent, though it might also be account-related…”

Jev: typed answers

Ticket: “I was charged twice this month!”

```
department: billing
priority:   high
complexity: tier1
```

WHAT I BUILT StreamWave ticket routing recipes/ticket-routing/go



Every step is durable. A Jev timeout is just an Activity retry.

PLOT TWIST

I was thinking way too small.

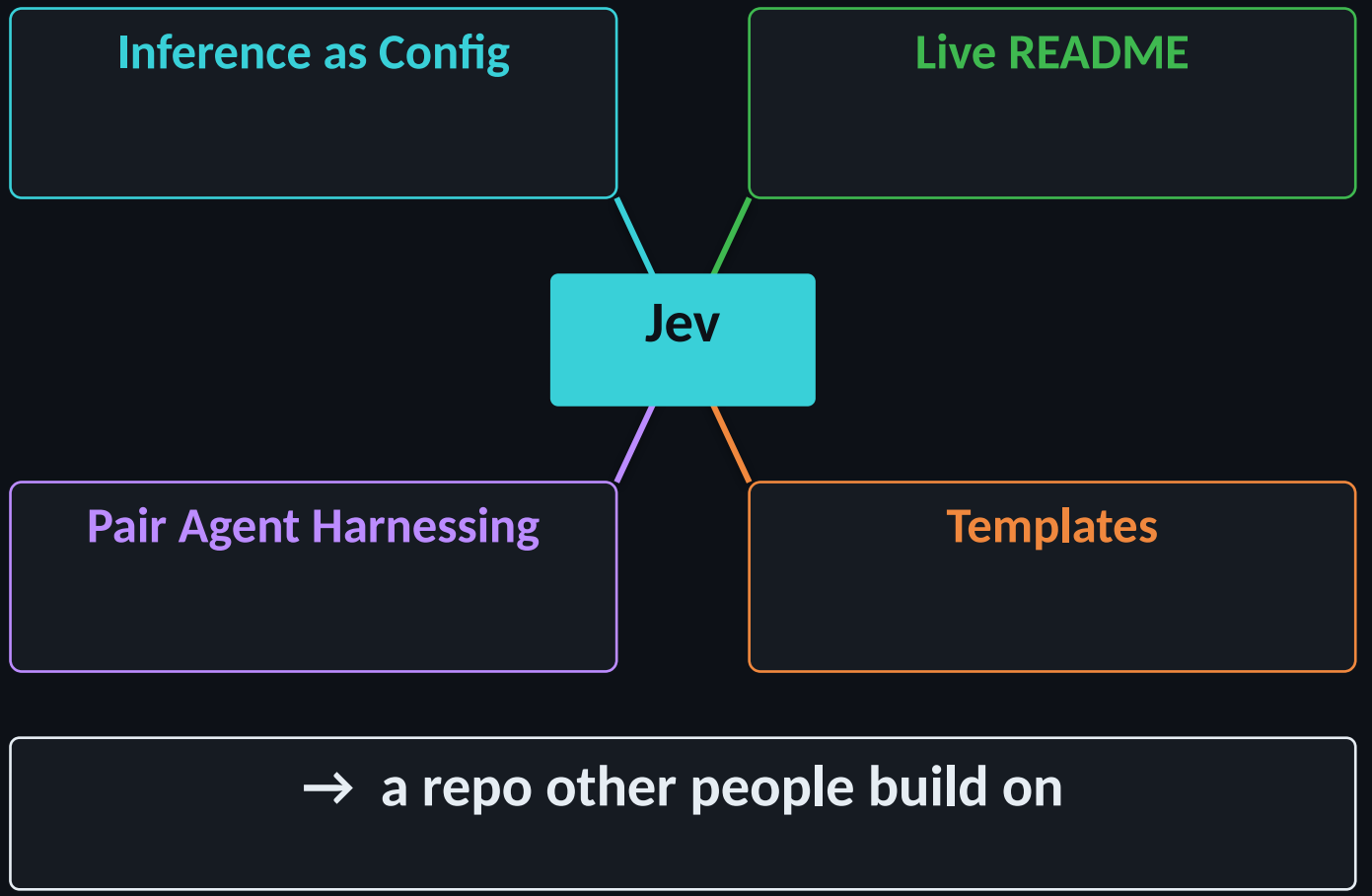
WHAT I THOUGHT I WAS BUILDING



One more Jev demo.

(there are already a million)

WHAT ACTUALLY HAPPENED



30 DAYS · 1 DAY = 1 SECOND

Everyone aimed at Jev.



DAY

00

SEP 15

0

Jev clones & rivals

Sep 15 **TypeSafe AI launches Jev**

WE PREPARED SOMETHING FOR YOU

cadence-ai-samples

A ready-to-run repo of common AI samples.

CADENCE-AI-SAMPLES

Skip the config headache.





Running in minutes.

CADENCE-AI-SAMPLES

Run any recipe your way

RECIPES	FRAMEWORKS	MODELS	CLASSIFIERS
Ticket routing	Google ADK	Gemini	Jev
Recurring watch	OpenAI Agents	OpenAI	Laya (local)
Human-in-the-loop		Llama (local)	
Agent handoffs			
Auto-research			

See it in action. Read the code. Adapt it to yours.

Four challenges, four solutions

CHALLENGE		SOLUTION
#1 Models change constantly	→	1 Inference as Config
#2 Setup takes an afternoon	→	2 Live README
#3 Coding agents forget everything	→	3 Pair Agent Harnessing
#4 Frameworks are a dependency trap	→	4 Templates over Frameworks

~/cadence-ai-samples 1

Inference as Config

Models and agents are swappable configuration, not hardcoded logic.

01

The Duct Tape Architecture

ticket-routing/workflow.go (v0, a reenactment)

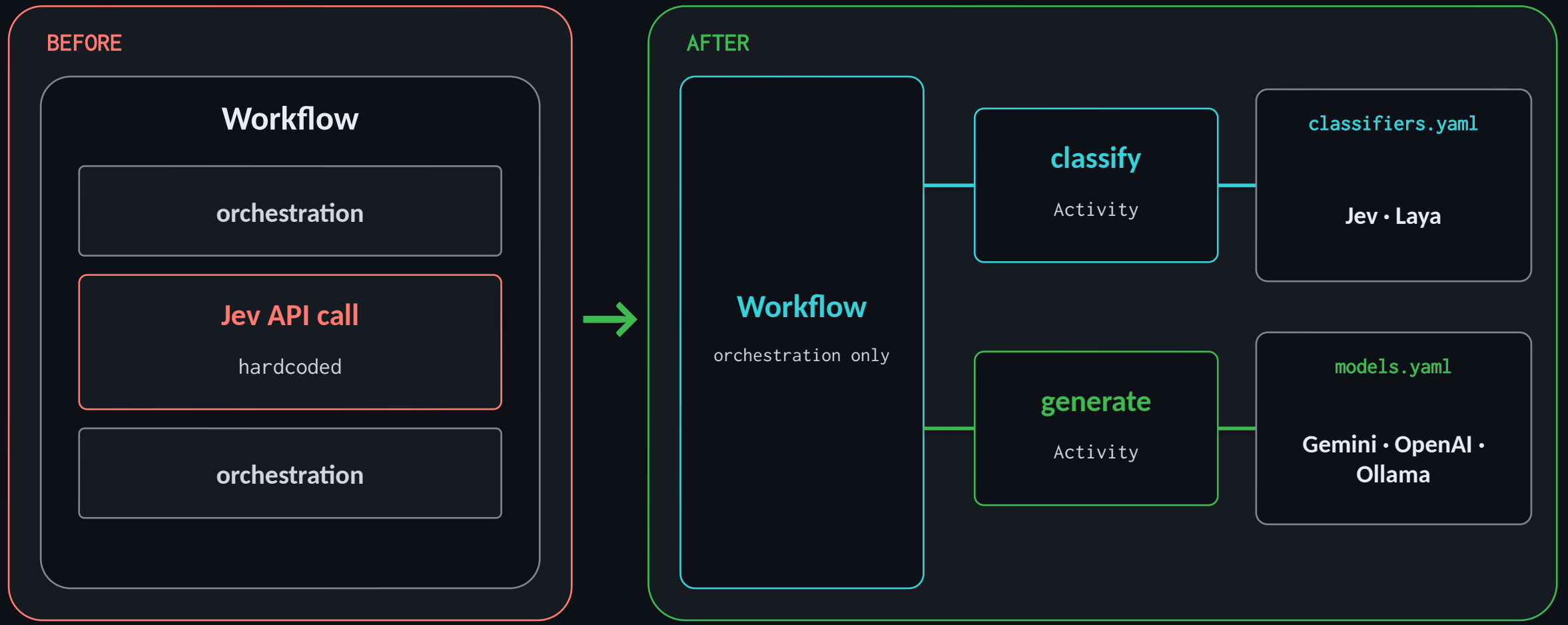
```
func RouteTicket(ctx workflow.Context, t Ticket) {
    // orchestration...
    resp := http.Post("https://api.typesafe.ai/...")
    // TODO: key? limits? local?
    queue := parse(resp)
    // more orchestration...
}
```

It worked!	Once.
429	Hit an API limit
“Run it locally?”	Nope, hardcoded
New model drops	Rewrite. Again.

Jev is awesome: structured decisions, not chatty paragraphs.

Stop hardcoding intelligence

Before: one model baked into the Workflow. After: composable parts.



Two YAML catalogs. Pick a key.

DECISION CLASSIFIERS

classifiers.yaml

```yaml
classifiers:
  - id: jev-default
    provider: typesafe
    model: jev-latest

  - id: laya-local
    provider: laya
    endpoint: localhost:8008
```

GENERATIVE MODELS

models.yaml

```yaml
models:
  - id: gemini-flash-lite
    provider: google

  - id: llama3.2-local
    provider: ollama

  - id: openai-nano
    provider: openai
```

```
$ main.py --classifier-id laya-local --model-id llama3.2-local
```

The plug-in matrix

Same orchestration code. Five different brains.

Mock	Cloud	Budget	Hybrid	Airplane
no keys + no models	jev-default + gemini-flash-lite	jev-default + openai-nano	jev-default + llama3.2-local	laya-local + llama3.2-local
Test the plumbing	Accuracy	Cost	Local generation	Fully offline

…and now type that by hand

```
.venv/bin/python main.py --domain cadence-ai-samples --task-list recurring-ai-
   watch-python-google-adk-live-llama3.2-local-laya-local --workflow-id recurring-ai-
   watch-python-google-adk-live-llama3.2-local-laya-local-demo --model-id llama3.2-
   local --classifier-id laya-local worker --mode live --confirm-live   # good luck
```

~/cadence-ai-samples 2

Live README

Turn documentation into an interactive, local execution guide.

02

The README Lie

README.md

```
## Recurring AI Watch

 1. Install deps        make bootstrap
 2. Copy the config     cp config.example.json config.json
 3. Check Laya          curl -s localhost:8008/health
 4. Check Ollama        curl -s localhost:11434/api/tags
 5. Set the model       export JEV_MODEL=jev-v1
 6. Find a model ID     grep "id:" models.yaml
 7. Find a classifier   grep "id:" classifiers.yaml
 8. Edit the workflow   vim workflow.py   # line 42
 9. Build the task list --task-list recurring-ai-watch-…-laya-local
10. Terminal 1          ./run.sh worker --domain … --task-list …
11. Terminal 2          ./run.sh start --model-id … --classifier-id …
12. Poke it             ./run.sh check-now
13. Hope                ./run.sh status

*beautifully formatted*
```

coworker.exe

has stopped responding

```
( o_o )
```

eyes: glazed

No one was going to do that.

Docs shouldn't be a text file

Clone-to-first-run friction was killing the project.

README.md

- ✗ Static text
- ✗ You type the flags
- ✗ You guess if services are up
- ✗ Goes stale quietly

Live README

- ✓ Reads your checkout
- ✓ Generates cmd line flags
- ✓ Checks your services
- ✓ Always current

If inference is just config, why are humans typing it?

~/cadence-ai-samples 2 · SOLUTION

The Recipe Explorer

```
$ ./scripts/
  explore.sh
```

Reads the repo

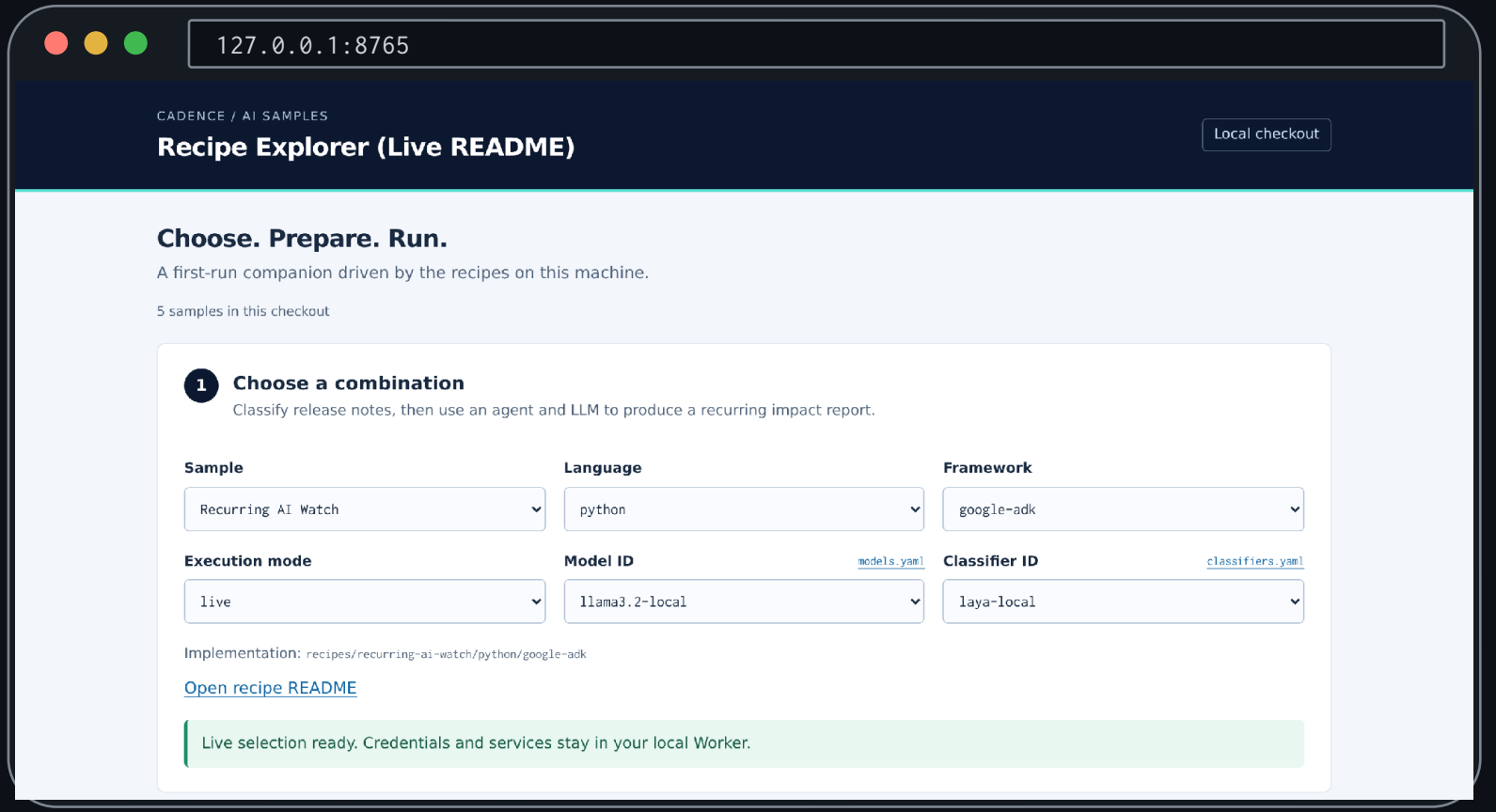
Recipes + YAML catalogs, local edits included

Validates deps

Checks services, warms local models

Writes the commands

Copy, paste, run



Recurring AI Watch



Fast + cheap decides.
Slow + smart only when needed.

The Workflow never names a model

recipes/recurring-ai-watch/python/google-adk/workflow.py

```
AI_ACTIVITY_OPTIONS = {
    "schedule_to_close_timeout": timedelta(seconds=30),
    "retry_policy": {
        "initial_interval": timedelta(seconds=1),
        "backoff_coefficient": 2.0,
        "maximum_attempts": 3, …

async def _check(self) -> None:
    update = SCRIPTED_RELEASES[self._next_update_index] …
    try:
        decision = await workflow.execute_activity(
            CLASSIFY_ACTIVITY,
            ClassificationDecision,
            update,
            **AI_ACTIVITY_OPTIONS,
        )
        if decision.warrants_report:
            self._latest_report = await self._report(update)
```

AI call = Activity

Timeouts, retries and backoff come from AI_ACTIVITY_OPTIONS.

Typed decision

ClassificationDecision, not a paragraph to parse.

No model in sight

Model and classifier IDs arrive as input from the catalog.

Jev and the model plug in at the Worker

live.py · the Jev / Laya classify Activity

```python
@activity.method(name=CLASSIFY_ACTIVITY)
def classify(self, update) -> ClassificationDecision:
    question = {"type": "choice",
        "instructions": "Does this release …",
        "criteria": {"yes": …, "no": …}}
    request = Request(self._endpoint, data=json.dumps({
        "state": f"Version: {update.version} …",
        "model": self._model,
        "questions": {"relevant": question},
    }).encode(), …)
```

inference.py · the report agent

```python
agent = LlmAgent(
    name="recurring_ai_watch",
    model=RetryingCadenceModel(
        model=runtime_model(provider, model_name)),
    instruction=INSTRUCTION,
)
runner = CadenceAgentRunner(
    app_name="recurring-ai-watch",
    agent=agent, session_service=sessions)
async for event in runner.run_async(…): …
```

main.py · Worker startup

```python
selection = load_selection(args.catalog_dir, model_id=args.model_id, classifier_id=args.classifier_id)
classifier, model_activities = live_activities(selection, os.environ)
registry = build_registry(classifier.classify, model_activities)
```

No model is hardcoded. Every recipe and template ships this way.

~/cadence-ai-samples 3

Pair Agent Harness Engineering

Humans direct agents through durable artifacts: Issues, PRs and specs.

03

The Multi-Machine Prompt Email Disaster



SPEC.md files everywhere

Zero visibility into what got built

“Which laptop has the latest?”

It was a complete disaster.

Stop relying on agent memory

Agent memory

- ✗ Lives in one chat, on one machine
- ✗ Fills up, forgets, drifts
- ✗ Nobody else can see it

Durable artifacts

- ✓ GitHub Issues: decisions
- ✓ Specs: intended behavior
- ✓ PRs + Git: what was built

Agents are ephemeral. Spin one up, do one job, throw it away.

Pair Agent roles

`AGENTS.md` defines every agent's role: start here, every time

You

Direct the work
Approve specs & PRs
Only humans approve

→

Pair Agent Wrangler

Manages the backlog
Grooms Issues
Reviews code

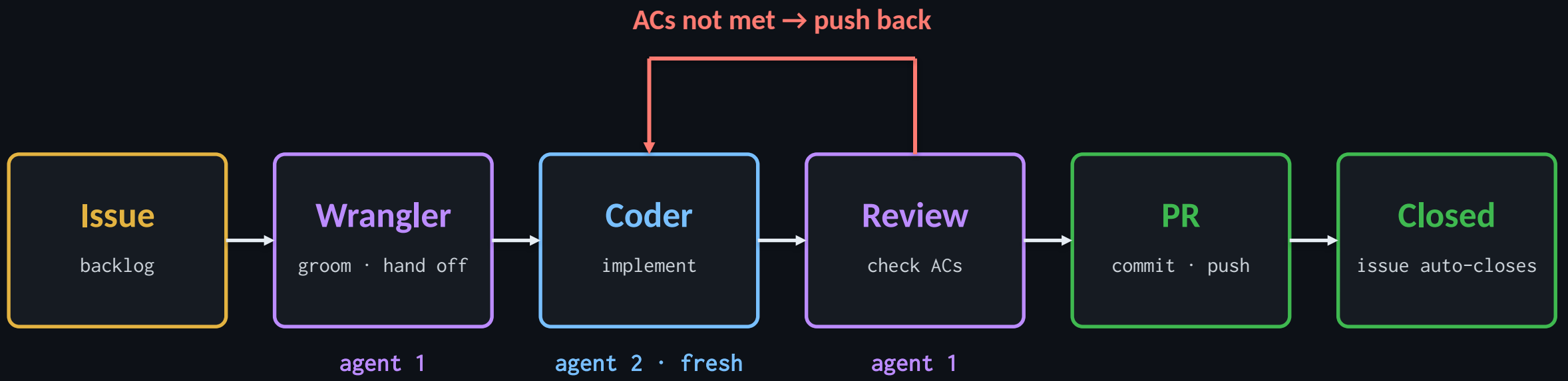
→

Coder agents

One scoped Issue
Implement, hand off
…then destroyed

Context stays clean.

The Pair Agent workflow



Context lives in the Issue, not in the agent.

The OSS bonus

2 laptops + 1 tired developer + prompts in email = my problem

my problem × N contributors = OSS problem

∴ durable artifacts solve both

solve for N

```
you> Proceed with Issue #N.
agent> Rechecked claim · labeled in-progress · created issue-N-short-slug
```

~/cadence-ai-samples 4

Templates, not another framework

Copy a folder. Run it. No shared dependencies.

04

The Framework Trap

AT UBER

We want teams rapidly productive. Yet another framework makes us the bottleneck, and the gatekeepers of new features.

ultimate-ai-framework

Docs	coming soon™
Dependencies	143
Written	one weekend
Maintainers	me, sometimes

WHY FRAMEWORKS HURT

- ✗ You're stuck maintaining it
- ✗ It can't keep up with new SDK features
- ✗ Shared dependencies break everyone
- ✗ You lose flexibility
- ✗ Docs never catch up

Templates over frameworks

```
$ ./scripts/new-recipe.sh invoice-review \
    --language python --classifier systemone --agent openai-agents
```

Recurring shapes › Profile system › **Standalone folder** › Your business logic › Copy into your app

WHY TEMPLATES WIN

- ✓ You own the code
- ✓ New SDK features on day one
- ✓ Zero shared dependencies
- ✓ Change anything
- ✓ README + SPEC generated with every recipe

10 profiles · 3 languages · 0 shared deps

One command, a complete recipe

```
$ ./scripts/new-recipe.sh invoice-review \
    --language python --classifier systemone --agent openai-agents
```

created

```
recipes/invoice-review/
├── README.md
├── SPEC.md
├── PROMPT.md
└── python/openai-agents-systemone/
    ├── workflow.py  main.py  config.py
    ├── classifier.py  inference.py  live.py
    ├── run.sh  pyproject.toml
    └── tests/test_profile.py
```

Docs

→ Live README

Your code

runnable · tested

JOIN IN

Open for contributions, just fork and...

cadence-ai-samples isn't finished. It's a foundation.

Writing code?

Try Pair Agent Harnessing.
Grab an **agent-ready** Issue:

```
you> Follow agents.md, Proceed with Issue #N.
```

Not coding today?

Tell me the real problem.
I'll groom it into an agent-ready spec:

```
you> Follow agents.md, Capture this idea: …
```

Contribute to this repo, or to Cadence itself.

LET'S TALK

Tell me why my Python code is terrible.

Grab an issue · Spin up a template · Build together

Discussions

GitHub
cadence-ai-samples

Contact us

cadenceworkflow.io/
community/contact-us

Slides

Download the PDF
cadenceworkflow.io/presentations/
2026.10.22-seattle-systems.pdf

Thank you!